CS 271P · Artificial Intelligence · Fall 2026

PA 4: Weighted Stable Garden

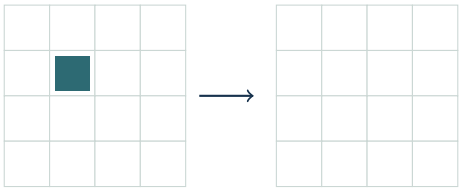
Submission Instructions

1. DESIGN A GARDEN THAT STAYS THE SAME

Imagine a garden divided into square tiles. You choose which tiles contain a plant and which stay empty. Some tiles are blocked, like paths or rocks, so you cannot plant there.

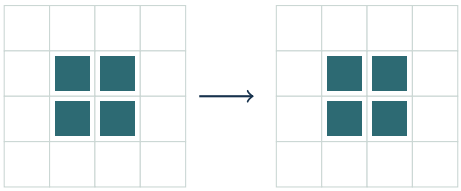
In this puzzle, plants respond to their neighbors. At the next update, an isolated plant disappears, while a new plant can appear on an empty tile with just the right number of nearby plants. Your job is to design a layout that **stays exactly the same after an update**. We call this a **stable** garden. The update rules are explained on the next page.

One plant disappears



An isolated plant has no neighboring plants.
This layout is not stable.

A block of four stays unchanged



Each plant has three neighboring plants.
This layout is stable.

Teal squares contain plants; white tiles are empty. Each arrow shows one update.

Each tile also has a positive number, its **weight**. A plant on that tile earns that many points of garden value. Your goal is to find a **stable garden with as much total value as possible**. A valuable tile is useful only when planting it is compatible with the stability rules.

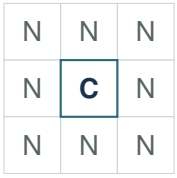
What you will make and submit

- Write a Python program that chooses a stable layout for each garden it receives.
- Work in the supplied `solver.py`; upload that one file to Gradescope.
- Your program is graded automatically: 100 ordinary points, plus up to 5 optional extra credit points.

Reading route: learn the rules on page 2, see the input and output on page 3, then start running the supplied files on page 4. Grading and submission are on page 6; optional extra credit is on page 7.

2. THE RULES: EVERY TILE MUST STAY UNCHANGED

A tile containing a plant is **live** (1); an empty tile is **dead** (0). Each tile has eight neighboring positions: above, below, left, right and the four diagonals. Count their live plants, using the layout *before* the update. The tile itself does not count.



C: tile being checked
N: one of its neighbors

The update rule is based on Conway's Game of Life:

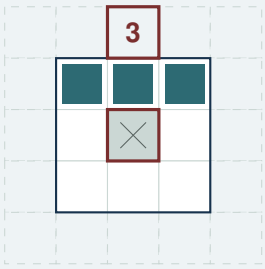
- Exactly **three** live neighbors: the tile is live after the update.
- Exactly **two**: it keeps its current state.
- Any other count: it is dead after the update.

All tiles update at the same time.

To stay unchanged, your chosen layout must satisfy both rules below:

Chosen state	Condition for staying in that state
Live (1)	It has exactly two or three live neighbors.
Dead (0)	It has any number of live neighbors *except* three.

Blocked tiles must be dead, and must obey the dead-tile rule. Simply leaving a blocked tile empty is insufficient if its neighbors would cause a plant to appear there. **Every position outside the garden is also dead and must stay dead.** Checking one surrounding row/column on each side is enough; farther positions cannot see a plant.



Check empty positions too.
Each red-outlined dead position sees the three plants in the top row. A plant would appear at both the blocked tile (gray, with a cross) and the outside position above it (dashed). Either birth makes this layout illegal.

The all-dead board is always legal and has value zero. It is a useful starting answer. For a valuable answer, you must satisfy *all* the rules while choosing where to plant.

3. WHAT YOUR FUNCTION RECEIVES AND RETURNS

The local tests and Gradescope call `solve(instance, budget, rng)` in `solver.py` once per garden. The starter already defines this function, and the test runner supplies all three arguments. `instance` describes one garden. `budget` gives you a timer and a way to save answers (page 5). `rng` is a supplied `random.Random` generator for any random choices your algorithm makes.

`instance`, the first argument to `solve`, is a dictionary with four fields:

`height`, `width`	Positive integers giving the numbers of rows and columns.
`blocked`	A list of rows of Booleans: `True` means planting is forbidden.
`weights`	A list of rows of positive integers: the value of a live plant on each tile.

Both matrices have `height` rows and `width` entries per row. Coordinates are zero-based: `board[i][j]` means row `i`, column `j`, with row 0 at the top. `instance` supplies weights and blocked tiles; your program chooses the entire live/dead layout.

Worked example: a 3-row, 4-column garden

```
instance = {
    "height": 3, "width": 4,
    "blocked": [[False, False, False, False],
                [False, False, False, False],
                [False, False, True,  False]],
    "weights": [[2, 7, 1, 4],
                [3, 5, 9, 2],
                [6, 1, 8, 3]],
}
# One legal board your function could return:
board = [[1, 1, 0, 0],
         [1, 1, 0, 0],
         [0, 0, 0, 0]]
```

Input garden

2	7	1	4
3	5	9	2
6	1	×	3

Legal answer: 17

2	**7**	1	4
3	**5**	9	2
6	1	×	3

Better answer: 22

2	**7**	**1**	4
3	**5**	**9**	2
6	1	×	3

Only live tiles contribute: the returned board has value $2 + 7 + 3 + 5 = 17$. Moving the block one column right gives $7 + 1 + 5 + 9 = 22$. Both are stable; the second is more valuable. This illustrates improvement, without claiming that 22 is the garden's maximum possible value.

> **Return a list of rows of ordinary Python integers, each 0 or 1.** Use the dimensions in `instance`. Convert Z3 values to integers before returning. Booleans such as `True`, tuples, NumPy arrays and Z3 expressions are rejected as output. A blocked tile's weight never contributes because it must remain dead.

4. GET SET UP, THEN TEST YOUR SOLVER

Use Python **3.10 or newer**. Extract the submitted-problem folder and open a terminal in it: you should see `solver.py`, `test_local.py` and `requirements.txt`. Run the commands below from that folder.

Step 1: install Z3; use the primer if Z3 is new to you.

```
python3 -m pip install -r requirements.txt
```

This installs the grading version of Z3; import it in Python as `z3`. **New to Z3?** Open `Z3 Primer.ipynb` and Run All (about 20 minutes). It teaches choices, rules, reading models and improving value through a small science-booth problem. You write the garden model yourself.

Then run `python3 z3_example.py`. This separate teaching example solves its own open 3×3 garden and prints a 0/1 board with value 6; **it does not run your** `solver.py`.

Step 2: test the saved `solver.py` **on one practice garden.**

```
python3 test_local.py --list
python3 test_local.py --case 4 --compare
```

`--list` displays numbered practice gardens and their sizes, without running your solver. The second command runs `solve` in your saved `solver.py` on case 4, an open 3×3 garden, checks the answer, and reports its value. `--compare` adds reference values. The unchanged starter places 2×2 blocks and returns a legal board of value 4 here. The exact maximum is 6, giving you a small first improvement to work on.

How to read the feedback

- **VALID**: your board passed the format, blocked-tile and stability checks.
- **Exact tiny optimum**: the maximum value is known for this small case. "Matched" means your answer reached it.
- **Public reachable target**: the value of a supplied stable layout on the same input garden. Your value can exceed it; it need not be optimal.

Ordinary public target comparisons help diagnose changes; they do not predict your grade on different private inputs.

Step 3: improve `solver.py`**, save it, and rerun the command for case 4.** This is the file you edit and submit. Extend or replace its `solve` function; aim for a legal value-6 answer here, then try broader tests (page 8). To check a candidate within your search, `problem.check_output(instance, board)` reports `valid`, `value` and `errors`. An error at a coordinate such as `(-1, 2)` refers to a position just outside the top edge.

You may use systematic search, local search, Z3, or a combination. **No particular search algorithm is required.** Z3 is the only external constraint solver allowed; your own search can use standard Python.

Optional: inspect a picture or try the notebook (about five minutes).

```
python3 -m pip install -r requirements-visual.txt
python3 test_local.py --case 4 --compare --draw
```

The draw command shows the input weights, a supplied stable layout, and your answer if it is valid. You can also open `Explore Garden.ipynb`, Run All, select a preset, and press **Run saved solver.py**. Preset selection performs no search. These visuals are optional; no notebook or screenshot is submitted.

5. SEARCH WITHIN THE TIME LIMIT AND SAVE ANSWERS

Your program gets **three seconds per garden, measured by elapsed clock time**. This includes preprocessing, building the model, search, checking and saving answers, and returning them. A solver timeout limits a solver call; the assignment timer covers everything.

Start with a legal fallback, then save improvements. The runner supplies these methods on `budget`:

`remaining()`	Seconds left in this case.
`expired()`	Whether the allowance has ended.
`offer(board)`	Check and save a legal board if it improves the saved value and time remains. Returns `True` if accepted, `False` otherwise.

Here is the starting/finishing structure, before adding your search:

```
import problem

def solve(instance, budget, rng):
    best = problem.empty_board(instance)
    budget.offer(best)  # Save a legal, zero-value fallback.
    # Add your search here. Keep best legal.
    # When best improves, call budget.offer(best).
    return best
```

At most **64 improving offers** are saved. Offer useful improvements, then return your best board before time expires. The grader uses the highest-value accepted offer or valid timely return. An accepted offer remains available even if a later return is late or invalid, or your code raises an exception. A fallback shown by the notebook is only a display; your file must produce or offer its own answer.

For a Z3 call, leave time to decode and save its answer. For a solver object named `solver`, a timeout can be set as follows:

```
remaining = budget.remaining() - 0.10  # Reserve return time.
if remaining > 0:
    solver.set(timeout=max(1, int(1000 * remaining)))
    status = solver.check()  # Z3 timeout units: milliseconds.
```

- `sat`: Z3 found an assignment satisfying this model. Decode its cells to integer 0/1 values and check the board before saving it.
- `unsat`: no assignment satisfies this model. If you asked for a *strictly better* value, this proves your current answer optimal within the modeled choices.
- `unknown` or a timeout: there is no such proof. Keep your best checked board.

Optimizing only a region with its surroundings fixed proves optimality only for those choices if the optimization completes. Include every rule affected by a change, including rules just outside the region.

Other limits: one search thread; 1 GiB of virtual address space in the Linux worker. Import has a separate **ten-second** elapsed-time allowance. Put your search inside `solve`, so importing the file only defines your code. Each case starts a fresh process and imports your current saved source.

6. GRADING AND SUBMITTING YOUR WORK

Your ordinary score is out of **100 automatic points**. Correctness and optimization are assessed separately:

Points	What earns them
30	**Legal answers.** 15 points are divided equally over the eight public fixed examples; 15 over private generated 5×6 gardens.
20	**Exact answers on tiny gardens.** Divided equally over the six public fixed examples with at most nine cells. Each must match its exhaustively verified maximum value.
50	**High-value answers on larger gardens.** Ten published garden categories have equal weight; cases within each category also have equal weight. See page 8 for the shapes.

What is the target value? For each larger quality test, staff save a **legal, stable layout** on the input garden: the same dimensions, blocked tiles and weights your solver receives. The sum of the weights of its live tiles is the **target value** T.

You choose your own stable layout on that garden; it is compared by value. For your legal answer with value V, the fraction of that case's quality points is:

$$\min\left(1, \frac{V}{0.98T}\right).$$

For a saved staff layout of value $T = 100$, your legal value 98 or higher earns all of that case's quality points; value 49 earns half. Invalid, missing or zero-value answers earn zero quality points.

The target is a **reachable value, not a proved maximum**. Staff make three independent Z3 runs with the student's search/import/thread limits and save the layout with the lowest checked value. Targets are frozen before grading and rechecked each run; grading uses saved results.

Your final grade uses the FINAL evaluation. Gradescope uses regular tests while submissions are open. After submissions close, all submissions are rerun with FINAL; its ordinary and bonus scores replace the earlier scores. FINAL has more and generally larger gardens:

Case group	Regular	FINAL
Generated validity cases	10	20
Quality cases	20 (2 per category)	40 (4 per category)

The eight fixed examples, rules, point weights, function and time limits stay the same. Public correctness feedback is visible; private quality feedback becomes visible after grades are published.

Before uploading to Gradescope

1. Save `solver.py`, with `solve(instance, budget, rng)` defined.
2. Run the ordinary and FINAL local suites (commands on page 8); inspect validity, tiny optima and larger-case value.
3. Keep your implementation and helpers in that one file. The supplied public `problem.py` is available to import; other files you create are not submitted.
4. Upload **the file** `solver.py` **itself**, using that exact name. Upload one file, without a ZIP, folder or notebook.

Submit the tutorial survey separately for Part 1. The post-submission paper debrief is formative and ungraded.

7. OPTIONAL EXTRA CREDIT: EXACT ANSWERS ON SMALL STRUCTURED GARDENS

The same `solver.py` can earn up to **5 additional automatic points**. The bonus gardens are small, and blocked rows and columns divide each one into rooms. On these gardens only an **exact** optimum earns credit. The grader checks your board's value; **any method that reaches the exact optimum earns the same bonus**. Submit no extra file or written proof.

A direction worth exploring. A garden with this kind of structure may break into pieces that can be solved separately and then put together. Whether that is ever safe under the stability rules, which pieces are safe to separate, and how to combine them, is for you to work out. Look at the public bonus gardens first, with `--bonus --list` and `--bonus --case 1 --draw`, and check every whole board you build. A whole-board method that reaches the exact optimum in time earns the same points.

Bonus	The two assessed families
4 pts	**Several small rooms.** Regular: six 4×4 rooms in a 10×16 board. FINAL: eight rooms in 10×22. Blocked rows and columns separate the rooms, and a room may contain blocked tiles.
1 pt	**Two rooms and a thin wall.** Two open 3×3 rooms with one blocked column between them: a 3×7 board. FINAL uses the same size.

Weights are 1–12; layouts may be transposed or reflected. Each family has two private cases in regular grading and four in FINAL. Its points are divided equally among its cases. A legal answer matching a case's exact optimum earns that case's points; suboptimal, invalid or missing output earns zero. Bonus points accumulate across cases and are added even if your ordinary score is below 100. The assignment total is 100; the reported maximum is 105.

These small-case optima are independently verified by exhaustive checking. The same three-second total search and other resource limits apply. Public bonus practice uses different exact inputs of the same families and sizes:

```
python3 test_local.py --bonus --list
python3 test_local.py --bonus --case 1 --compare
python3 test_local.py --bonus
python3 test_local.py --bonus --final
```

The full bonus practice suite reports a score out of five; a single-case run reports whether you matched that case's exact optimum. This is practice feedback, not a prediction of your private bonus grade.

8. REFERENCE: LARGER TESTS AND SUBMISSION RULES

All cases use the same stability rules and input format. Only the geometry, blocked tiles and weights vary. The following quality categories are published so you can build a general solution for their different shapes.

Category	Regular size	FINAL size
Scattered blocked tiles	18×20	22×24
Two-cell-wide blocked bands	18×20	22×24
Long strip	36×4	44×5
Open uniform rectangle	$20–22 \times 22–24$	$24–26 \times 26–28$
Irregular separated islands	18×20	22×24
One-cell-wide thin walls	18×20	22×24
Connected rooms with corridors	18×20	22×24
High-weight hotspots	18×20	22×24
Wider strip	36×6	44×7
Mixed separated narrow rooms	26×30	30×34

Sizes may be transposed and patterns reflected. Most weights are 1–12; hotspots can reach 301; open rectangles have uniform weights. No category introduces a new rule, negative weight or mandatory live tile. Your input has no category label or target value.

Run broader checks after a focused test works.

```
python3 test_local.py --compare          # 18 regular cases
python3 test_local.py --final --compare  # 28 FINAL cases
python3 test_local.py --case 18 --compare  # One larger case
python3 test_local.py --json results.json  # Save local feedback
```

Both ordinary suites contain the same eight fixed examples, plus separate representative larger cases with checked comparison boards. Case numbers start at 1. Use `--list` with `--final` to inspect the FINAL catalog.

Optional flag	What it changes
`--module path/to/solver.py`	Tests another file instead of the saved starter path.
`--seed N`	Repeats the solver's random seed; practice inputs themselves are fixed.
`--seconds S`	Sets an experimental local time limit; grading uses three seconds.
`--draw`	Plots a selected `--case N`; needs the optional visual dependencies.
`--bonus`	Selects the separate bonus suite, with four regular or eight FINAL cases.

What belongs in your submitted program. Use a general implementation in `solver.py`. You may import standard Python modules, the supplied `problem.py`, and `z3`. Do not edit supplied helpers, use another external constraint solver, read private grader files, access the network, invoke subprocess solvers, or rely on auxiliary files or precomputed answer databases. Small reusable stable shapes are fine; do not hardcode complete boards for test inputs.

> **The goal: correct constraints, useful search, and a legal high-value answer.** A local **VALID** result confirms legality. Exact tiny results confirm the maximum on those tiny cases. Larger public comparisons give evidence about value on those inputs. Use each kind of feedback for what it measures.